A Useless Recipe for Primes: A Partition-Based Algorithm for Prime Synthesis

Bruno Ziki Kongawi
microgeant@gmail.com

June 16, 2026

Abstract

Prime generation algorithms fall into two classical families: sieving methods, such as the Sieve of Eratosthenes, which systematically eliminate composites; and primality testing of individual candidates. We introduce a third, lateral approach: partition-based constructive synthesis. By applying algebraic operations to binary set partitions and exponent vectors, we demonstrate a method for synthesizing primes directly from a seed set of their predecessors. We enumerate the algorithm's configurations precisely and establish its complexity as $O(N \cdot (2E)^N)$, confining the method to theoretical and recreational use. Crucially, we identify a deterministic 'Safe Synthesis Zone'—a quadratic search window $(m_k, m_k^2]$ within which every synthesized candidate is necessarily prime. This framework provides new theoretical insights into the additive-multiplicative structure of integers, suggesting that primes can be fundamentally apprehended as a constructive, emergent property of its own history.

1 Introduction

1.1 Motivation and Scope

The distribution of prime numbers has fascinated mathematicians since antiquity, yet the methods for finding them have remained largely unchanged in spirit. For centuries, the hunt for primality has resembled the work of a sculptor, carving away the 'excess stone' of composites from the number line to reveal the structure beneath. This subtractive legacy defines primes primarily by what they are not.

The present work asks a different question: can primes be *constructed* directly without reference to the composite numbers they are defined to exclude?

We answer this affirmatively, albeit at a prohibitive computational cost. The resulting algorithm serves as a theoretical instrument that exposes a hidden, constructive symmetry in the additive-multiplicative anatomy of the prime sequence.

1.2 Euclid's Foundational Propositions

It is first observed that "*if two numbers be prime to one another, the sum will also be prime to each of them.*"[1]

Let p_1 and p_2 be prime to one another (i.e., their greatest common divisor is $\gcd(p_1, p_2) = 1$). Then their sum $p_3 = p_1 + p_2$ is necessarily prime to p_1 and p_2 (i.e., $\gcd(p_1, p_3) = 1$ and $\gcd(p_2, p_3) = 1$). In modern terminology, the set $\{p_1, p_2, p_3\}$ is pairwise coprime.

If $p_3 = p_1 + p_2$, then it follows from Proposition 24 of Euclid VII [1] that each of the following pairs is prime to one another (or coprime):

[1] See Proposition 28 of Euclid VII [1].

$$p_1 \text{ and } p_2 \times p_3$$
$$p_2 \text{ and } p_1 \times p_3$$
$$p_3 \text{ and } p_1 \times p_2$$

Then, by virtue of Proposition 28, each of the following sums is prime to p_1, p_2, and p_3:

$$\sigma_1 = p_1 + (p_2 \times p_3)$$
$$\sigma_2 = p_2 + (p_1 \times p_3)$$
$$\sigma_3 = p_3 + (p_1 \times p_2)$$

Let $p_4 = \sigma_1$. Then, similarly, each of the following unique sums is prime to the set $\{p_1, p_2, p_3, p_4\}$:

$$p_1 + (p_2 \times p_3 \times p_4)$$
$$p_2 + (p_1 \times p_3 \times p_4)$$
$$p_3 + (p_1 \times p_2 \times p_4)$$
$$p_4 + (p_1 \times p_2 \times p_3)$$
$$(p_1 \times p_2) + (p_3 \times p_4)$$
$$(p_1 \times p_3) + (p_2 \times p_4)$$
$$(p_1 \times p_4) + (p_2 \times p_3)$$

1.3 Generalization

Definition 1 (Binary Partition)**.** A *binary partition* of a set S is an ordered pair (A, B) where $A, B \subseteq S$, $A \cap B = \emptyset$, $A \cup B = S$, and $A, B \neq \emptyset$.

Thus, Euclid's proposition can be generalized as follows: given a seed set of pairwise coprime numbers $\{p_1, \ldots, p_n\}$, any sum S formed by a binary partition (A, B) of the set into two products—$S = (\prod_{i \in A} p_i) + (\prod_{j \in B} p_j)$—is necessarily coprime to every element in the set. This property is invariant under exponentiation, i.e., the pairwise coprimality of the resulting sums is preserved if each element in the seed set is raised to a positive integer power before partitioning.

Furthermore, by the second part of Euclid VII-28—"*...and, if the sum of two numbers be prime to any one of them, the original numbers will also be prime to one another*"[1]—it follows that if a number is prime to one of its parts, it is also prime to the remainder. Thus, for any coprime p_1 and p_2, the absolute difference $|p_1 - p_2|$ is necessarily coprime to both original terms.

Definition 2 (Powered Product)**.** For a subset $A = \{a_1, \ldots, a_k\} \subseteq P$ and an exponent vector $\mathbf{e} \in \{1, \ldots, E\}^k$, the *powered product* is defined as:

$$\pi(A, \mathbf{e}) = \prod_{i=1}^{k} a_i^{e_i}$$

2 The Algorithm in a Nutshell

Given a seed set of pairwise coprime numbers $P = \{p_1, \ldots, p_n\}$, a pair of powered products is obtained from a binary partition (A, B) and respective exponent vectors $\mathbf{e}_A$ and $\mathbf{e}_B$:

$$\pi(A, \mathbf{e}_A) = \prod_{p_i \in A} p_i^{e_i}, \quad \pi(B, \mathbf{e}_B) = \prod_{p_j \in B} p_j^{e_j}$$

New coprime numbers are then synthesized through the following algebraic operations:

$$\sigma_{\text{sum}} = \pi(A, \mathbf{e}_A) + \pi(B, \mathbf{e}_B), \quad \sigma_{\text{diff}} = |\pi(A, \mathbf{e}_A) - \pi(B, \mathbf{e}_B)|$$

2.1 Primes as a Special Case Through an Iterative Synthesis Procedure

To demonstrate the constructive nature of the framework, we define an iterative algorithm that expands a seed set P into an increasingly large set of pairwise coprime integers. We initialize the process with the fundamental seed set $P_0 = \{1, 2\}$ and a global exponent bound E. For any iteration $k \geq 0$, the next element p_{k+1} is synthesized through the following recursive procedure:

1. **Partitioning and Powering:** Generate the set of all binary partitions (A, B) of P_k. For each partition, compute the set of all possible powered products $\pi(A, \mathbf{e}_A)$ and $\pi(B, \mathbf{e}_B)$, where the components of the exponent vectors satisfy $1 \leq e_i \leq E$.

2. **Candidate Synthesis:** Construct the candidate set Σ_k by calculating the sums and absolute differences of all pairs of powered products:

$$\Sigma_k = \{\pi(A, \mathbf{e}_A) \pm \pi(B, \mathbf{e}_B) : (A, B) \in \text{Part}(P_k),\ \mathbf{e}_A, \mathbf{e}_B \in \{1, \ldots, E\}^*\}$$

3. **Local Filtering:** We define a search window relative to the current maximum element, $m_k = \max(P_k)$. We retain only those candidates $\sigma \in \Sigma_k$ that satisfy:

$$m_k < \sigma \leq m_k^2$$

4. **Selection and Adjunction:** From the accumulated filtered candidates, we select the minimal value $p^* = \min\{\sigma \in \Sigma_k : \sigma \notin P_k\}$. The set for the subsequent iteration is then formed by adjunction: $P_{k+1} = P_k \cup \{p^*\}$.

Algorithm 1 Iterative Prime Synthesis

Require: Seed set $P_0 = \{1, 2\}$, Exponent bound E, Iterations N

for $k = 0$ **to** $N - 1$ **do**
  $m_k \leftarrow \max(P_k)$
  $\Sigma_k \leftarrow \emptyset$ {Initialize candidate set}
  for all unique binary partitions (A, B) of P_k **do**
    for all exponent vectors $\mathbf{e} \in \{1, \ldots, E\}^{|P_k|}$ **do**
      $\pi_A \leftarrow \prod_{p_i \in A} p_i^{e_i}$
      $\pi_B \leftarrow \prod_{p_j \in B} p_j^{e_j}$
      for all $\sigma \in \{\pi_A + \pi_B, |\pi_A - \pi_B|\}$ **do**
        if $m_k < \sigma \leq m_k^2$ **then**
          $\Sigma_k \leftarrow \Sigma_k \cup \{\sigma\}$
        end if
      end for
    end for
  end for
  $p^* \leftarrow \min\{\sigma \in \Sigma_k : \sigma \notin P_k\}$
  $P_{k+1} \leftarrow P_k \cup \{p^*\}$ {Adjunction of minimal candidate}
end for
return P_N

We observe that the quadratic search window $(m_k, m_k^2]$ is inherently composite-free for any value coprime to P_k, provided P_k is a complete set of primes (i.e., all primes less than or equal to m_k are present in the set)[2]. Consequently, we posit that for a sufficiently large exponent bound E, the algorithm synthesizes enough contiguous primes at each iteration to maintain a gapless seed set, where every adjoined candidate is necessarily prime.

[2]If P_k contains all primes up to m_k, then the smallest composite number coprime to P_k must be p_{next}^2. Since $p_{next} > m_k$, it follows that $p_{next}^2 > m_k^2$ (or more precisely, $p_{next}^2 \geq (m_k + 1)^2$).

3 Witnessing the Synthesis: A Numerical Demonstration

Initializing with the seed set $P_0 = \{1, 2\}$, we define an adaptive exponent bound E_k for each iteration k. We set $E_0 = 1$ and increment this bound by 1 at each subsequent step, subject to a global constraint $E \leq 5$.

3.1 First Iteration

With only two elements, there is a single unique binary partition:

$$A = \{1\}, \quad B = \{2\}$$

At $k = 0$, the exponent bound $E_0 = 1$ renders the synthesis trivial, yielding only one combination of exponent vectors:

Exponents $(\mathbf{e}_A, \mathbf{e}_B)$	$\pi(A, \mathbf{e}_A)$	$\pi(B, \mathbf{e}_B)$	Sum	Difference
$(1, 1)$	$1^1 = 1$	$2^1 = 2$	3	1

The resulting candidate set is $\Sigma_0 = \{1, 3\}$. Applying the local filtering rule $m_0 < \sigma \leq m_0^2$ (or $2 < \sigma \leq 4$), we retain only $\{3\}$. Following the selection rule, we adjoin the minimal value $p^* = 3$, expanding our seed set to $P_1 = \{1, 2, 3\}$.

3.2 Second Iteration

With the seed set $P_1 = \{1, 2, 3\}$, we have three unique binary partitions:

1. $A = \{1\}, \quad B = \{2, 3\}$
2. $A = \{2\}, \quad B = \{1, 3\}$
3. $A = \{3\}, \quad B = \{1, 2\}$

At $k = 1$, our exponent bound is $E_1 = 2$. We examine the synthesis across these partitions. For brevity, we list only those candidates that fall within the quadratic search window $(3, 9]$:

Part.	$\mathbf{e}_A$	$\mathbf{e}_B$	$\pi(A, \mathbf{e}_A)$	$\pi(B, \mathbf{e}_B)$	Sum	Difference
1	(1)	$(1, 1)$	1	$2 \times 3 = 6$	7	5
2	(1)	$(1, 1)$	2	$1 \times 3 = 3$	5	1
3	(1)	$(1, 1)$	3	$1 \times 2 = 2$	5	1

The candidate set filtered by the search window $(3, 9]$ is $\Sigma_1 = \{5, 7\}$. Both are prime, in accordance with our observation that the quadratic window remains composite-free. Following the selection rule, we adjoin the minimal value $p^* = 5$, resulting in the expanded seed set $P_2 = \{1, 2, 3, 5\}$.

3.3 Third Iteration

With the seed set $P_2 = \{1, 2, 3, 5\}$, we have $2^{4-1} - 1 = 7$ unique binary partitions. The current maximum is $m_2 = 5$, establishing a search window of $(5, 25]$. At $k = 2$, the exponent bound increments to $E_2 = 3$.

For illustrative purposes, we examine several partitions using minimal exponents ($e_i = 1$). The resulting synthesis yields the following candidates:

Partition (A, B)	$\pi(A, \mathbf{1})$	$\pi(B, \mathbf{1})$	Sum	Difference
$(\{1,5\},\{2,3\})$	5	6	11	1
$(\{1,3\},\{2,5\})$	3	10	13	7
$(\{3\},\{1,2,5\})$	3	10	13	7
$(\{1,2\},\{3,5\})$	2	15	17	13
$(\{2\},\{1,3,5\})$	2	15	17	13
$(\{5\},\{1,2,3\})$	5	6	11	1
$(\{1\},\{2,3,5\})$	1	30	31	29

Filtering the results through the window $(5, 25]$, we obtain the set of candidate primes $\Sigma_2 = \{7, 11, 13, 17\}$. Each of these synthesized values is prime, verifying the composite-free nature of the quadratic window. Following the selection rule, we identify the minimal value:

$$p^* = \min\{7, 11, 13, 17\} = 7$$

By adjunction, the seed set for the fourth iteration becomes $P_3 = \{1, 2, 3, 5, 7\}$.

We note that while $E_2 = 3$ permits higher powers (such as $1 + 2^3 = 9$), the quadratic filter effectively removes composites like 9 in subsequent iterations if a smaller prime is not yet present, or if they fall outside the current window. In this instance, 9 is rejected as P_2 already contains its potential factor 3.

3.4 Fourth Iteration

Our seed set has grown to $P_3 = \{1, 2, 3, 5, 7\}$, and the adaptive exponent bound has reached $E_3 = 4$. The current maximum element $m_3 = 7$ defines a quadratic search window of $(7, 49]$.

With five elements in the set, there are $2^{5-1} - 1 = 15$ unique binary partitions. For brevity and pragmatism, we examine several representative partitions using unit exponents ($e_i = 1$) to identify the next prime candidate:

Partition (A, B)	$\pi(A, \mathbf{1})$	$\pi(B, \mathbf{1})$	Sum	Difference
$(\{1,2,5\},\{3,7\})$	10	21	31	11
$(\{1,3,5\},\{2,7\})$	15	14	29	1
$(\{1,5,7\},\{2,3\})$	35	6	41	29
$(\{1,2,3,5\},\{7\})$	30	7	37	23
$(\{1,2,3,7\},\{5\})$	42	5	47	37
$(\{1,2,7\},\{3,5\})$	14	15	29	1
$(\{1,3,7\},\{2,5\})$	21	10	31	11

Filtering the results through the search window $(7, 49]$, we obtain a set of prime candidates $\Sigma_3 = \{11, 23, 29, 31, 37, 41, 47\}$. We observe that every synthesized value in this window is prime. Applying the selection rule to find the minimal candidate:

$$p^* = \min\{11, 23, 29, 31, 37, 41, 47\} = 11$$

By adjunction, the seed set for the fifth iteration is $P_4 = \{1, 2, 3, 5, 7, 11\}$.

This iteration demonstrates the synthetic nature of the procedure: a single partition pair, such as $(\{1, 2, 5\}, \{3, 7\})$, simultaneously yields two primes (11 and 31). This suggests the existence of an arithmetic fulcrum—a point defined by the partition products around which primes are balanced through their sum and difference. As P grows, the algorithm does not merely find the next prime; it populates the quadratic window with a sequence of primes, often through multiple distinct partition paths, thereby providing a robust framework for the adjunction of successive seeds.

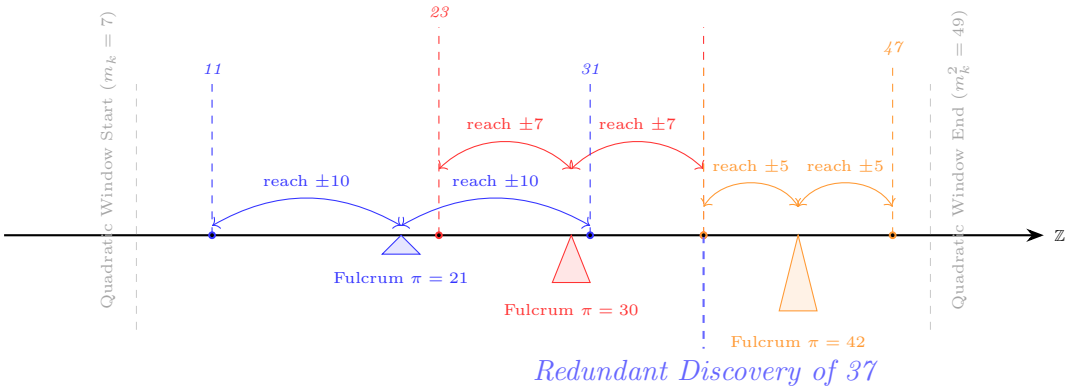

Figure 1: Superimposed synthesis paths: Multiple arithmetic fulcrums work in concert to populate the quadratic window. Note how prime 37 is discovered (over-determined) by two distinct but converging partition paths (fulcrums 30 and 42).

3.5 The Iterative Results

Each iteration yields an increasing density of primes within the search window:

Iteration	Seed Set (P_k)	Primes Discovered (Σ_k)
1	$\{1, 2\}$	3
2	$\{1, 2, 3\}$	5, 7
3	$\{1, 2, 3, 5\}$	7, 11, 13, 17, 19, 23
4	$\{1, 2, 3, 5, 7\}$	11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47
5	$\{1, 2, 3, 5, 7, 11\}$	13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97, 101, 103, 107, 109, 113

By the conclusion of the fifth iteration, the algorithm has successfully synthesized every prime number in the sequence from 2 up to 113. While many primes are "over-determined"—appearing through multiple distinct partitions and exponent combinations—the resulting set P_5 constitutes a perfectly gapless sequence of the first 30 primes.

The reach and density of the synthesis can be controlled by adjusting the global exponent bound E and the iteration count. For instance, if we maintain a constant bound $E = 2$ over ten iterations, we generate a robust collection of candidates. In this configuration, the algorithm continues to synthesize the prime sequence contiguously up to 167.

The first "gap" in the sequence occurs at the prime 173, which remained unsynthesized given the constrained search space. However, the algorithm's constructive power is evidenced by its discovery of much larger "islands" of primality; within these same ten iterations, the recipe synthesized primes as large as 1,319 (the 213th prime). This suggests that while $E = 2$ was insufficient to bridge the gap to 173, the combinatorial space of partitions remains a fertile ground for prime generation.

4 The Combinatorial Explosion: An Exponential Barrier

The constructive power of the synthesis framework comes with a significant computational cost. To understand the limits of the algorithm, we must calculate the number of configurations examined during a single iteration.

Table 1: Collection of Primes Synthesized in Ten Iterations ($E = 2$)

Synthesized Candidates (Total: 148)									
2	3	5	7	7	11	13	17	19	23
11	13	17	19	23	29	31	37	41	43
47	13	17	19	23	29	31	37	41	43
47	53	61	67	71	73	79	83	89	97
101	103	107	109	113	17	19	23	29	41
43	47	53	59	61	67	73	79	83	97
101	103	109	113	127	131	137	149	151	163
167	19	29	31	41	43	61	71	73	83
97	103	107	113	137	139	149	157	179	181
191	197	199	257	263	271	281	23	31	41
59	79	83	89	97	103	127	151	179	199
211	223	227	233	251	263	281	313	347	37
53	107	139	157	181	257	443	491	499	43
113	149	197	691	727	1069	1117	1237	1289	1319

4.1 Counting Partitions

For a seed set P of size n, we determine the number of ways to form a binary partition (A, B). Since each element has two choices (set A or set B), there are 2^n possible assignments. Excluding the cases where one set is empty leaves $2^n - 2$ ordered pairs. Because addition and the absolute difference are invariant to the ordering of the sets, we consider only unordered partitions.

Lemma 1 (Binary Partitions)**.** *The number of unique binary partitions of an n-element set is given by the Stirling number of the second kind:*

$$S(n, 2) = 2^{n-1} - 1$$

4.2 Counting Configurations

For a given partition, the number of exponent vectors $(\mathbf{e}_A, \mathbf{e}_B)$ is determined by the bound E. Since each of the n elements is assigned an exponent from $\{1, \ldots, E\}$, there are E^n possible vector combinations.

Theorem 2 (Configuration Count)**.** *For a seed set of size n and an exponent bound E, the total number of synthesized candidates $|\mathcal{C}|$ in a single iteration is:*

$$|\mathcal{C}(n, E)| = 2(2^{n-1} - 1) \cdot E^n \sim 2(2E)^n$$

While this characterizes a single step, the cumulative complexity of the recursive procedure over N iterations is dominated by the final term of the summation:

$$T_{\text{total}}(N, E) = \sum_{k=2}^{N} O(k \cdot (2E)^k) = O(N \cdot (2E)^N)$$

This asymptotic bound illustrates the "Price of Magic": every new prime discovered effectively doubles the effort required to synthesize its successor, ensuring the algorithm remains a theoretical curiosity rather than a practical competitor to the Sieve of Eratosthenes.

5 Conclusions

This paper has presented a constructive framework for prime generation rooted in the additive property of coprimality. By generalizing Euclid's Propositions VII-24 and VII-28 into a

partition-based algorithm, we have demonstrated that primes can be synthesized as algebraic functions of their predecessors. Our results suggest that:

1. **Constructive Primality:** Primes emerge naturally from the combinatorial interaction of partitions and exponents, without the need for traditional sieve-based elimination.
2. **Computational Thresholds:** The exponential complexity, $O((2E)^n)$, represents a fundamental barrier that characterizes the algorithm as a theoretical or recreational instrument rather than a practical generator.

While the Sieve of Eratosthenes remains the superior tool for large-scale prime discovery, this partition-based approach illuminates the deep, recursive structure of the prime sequence. It raises several intriguing questions for further exploration:

- **The Exponent Conjecture:** Does there exist a universal bound E, or a mapping $E(k)$, that guarantees the synthesis of the successive prime p_{k+1} for all k? Furthermore, can we identify an optimization strategy to determine the specific exponent vectors that uniquely target candidates within the search window for a given partition?
- **Combinatorial Density:** Which primes are inherently "over-determined" (reachable via a high multiplicity of partitions), and which exist at the sparse edge of the synthesis space?
- **Twin Prime Synthesis:** Within this framework, can we establish a bound for E that guarantees the discovery of twin prime pairs? Specifically, does every twin prime pair $(p, p+2)$ possess an additive-multiplicative representation as a partition-powered sum and difference?

The synthesis framework suggests a fundamental shift in perspective: prime numbers are not merely chaotic residuals defined by the exclusion of composites, but obey deterministic laws intrinsic to their own structure. By synthesizing primes directly through partitions, we demonstrate that they can be apprehended constructively, independent of their derivatives—the composites.

Ultimately, this "useless recipe" reveals the number line not as a sequence to be filtered, but as a structure to be built—demonstrating that computational elegance and practical efficiency are often distinct mathematical virtues. Whether this perspective yields further theoretical dividends remains an open and inviting question.

A Appendix: Algorithmic Implementation

The following Kotlin implementation serves as a functional reference for the Iterative Synthesis Procedure.

During the numerical demonstration, it was observed that as the seed set grows, partition products quickly exceed the capacity of standard 64-bit integer types. To preserve the Euclidean coprimality guarantee, all computations were performed using arbitrary-precision arithmetic (BigInteger). Failure to account for integer overflow results in the synthesis of 'pseudoprimality' artifacts—composites that share factors with the seed set due to bitwise wrap-around.

```
import java.math.BigInteger
import kotlin.collections.indices
import kotlin.sequences.toList

/**
 * Generates all unique binary partitions of a set.
 */
```

```kotlin
private fun <T> binaryPartitions(list: List<T>): Sequence<Pair<List<T>, List<T>>> =
    kotlin.sequences.sequence {
        if (list.size < 2) return@sequence
        val first = list[0]
        val rest = list.subList(1, list.size)
        val numPartitions = 1L shl rest.size

        for (i in 1 until numPartitions) {
            val left = mutableListOf<T>()
            val right = mutableListOf<T>(first)
            for (j in rest.indices) {
                if ((i shr j) and 1L == 1L) left.add(rest[j]) else right.add(rest[j])
            }
            yield(left to right)
        }
    }

/**
 * Generate all exponent combinations (odometer approach)
 */
private fun exponentCombinations(n: Int, maxExp: Int): Sequence<List<Int>> =
    kotlin.sequences.sequence {
        if (n == 0) return@sequence
        val indices = IntArray(n) { 1 }
        while (true) {
            yield(indices.toList())
            var i = n - 1
            while (i >= 0 && indices[i] == maxExp) {
                indices[i] = 1
                i--
            }
            if (i < 0) break
            indices[i]++
        }
    }

/**
 * Synthesizes primes using arbitrary-precision arithmetic to prevent overflow.
 */
private fun computePrimes(seeds: List<BigInteger>, maxExp: Int): Sequence<BigInteger> {
    if (seeds.isEmpty()) return kotlin.sequences.emptySequence()
    val maxP = seeds.maxOrNull() ?: return kotlin.sequences.emptySequence()
    val minR = maxP.add(BigInteger.ONE)
    val maxR = maxP.multiply(maxP)

    return sequence {
        for ((left, right) in binaryPartitions(seeds)) {
            val totalSize = left.size + right.size
            for (expVector in exponentCombinations(totalSize, maxExp)) {
                val leftExps = expVector.take(left.size)
                val rightExps = expVector.takeLast(right.size)

                val leftProd = left.zip(leftExps).map { (p, e) -> p.pow(e) }
                    .reduce { acc, b -> acc.multiply(b) }
                val rightProd = right.zip(rightExps).map { (p, e) -> p.pow(e) }
                    .reduce { acc, b -> acc.multiply(b) }

                val sumSigma = leftProd.add(rightProd)
                if (sumSigma >= minR && sumSigma <= maxR) yield(sumSigma)

```

```
                val diffSigma = leftProd.subtract(rightProd).abs()
                if (diffSigma >= minR && diffSigma <= maxR) yield(diffSigma)
            }
        }
    }
}

fun main() {
    var currentMaxExp = 1
    val maxIterations = 5
    var current = listOf(BigInteger.ONE, BigInteger.valueOf(2))
    var accPrimes = emptyList<BigInteger>()

    for (i in 1..maxIterations) {
        val found = computePrimes(current, currentMaxExp++).toList()
        val distinctFound = found.sorted().distinct()

        val currentSet = current.toSet()
        val diff = distinctFound.filter { it !in currentSet }
        val next = if (diff.isEmpty()) current else current + diff.min()

        accPrimes = accPrimes + distinctFound
        current = next
    }
    println("Discovered primes: ${(listOf(BigInteger.valueOf(2)) + accPrimes).
    distinct().sorted()}")
}
```

Listing 1: The Synthesis Engine using BigInteger

With a slight modification to the main function, we can run the algorithm for ten iterations with a fixed exponent bound of $E = 2$. This configuration allows us to observe the redundancy of the synthesis paths, generating the multiset of primes discussed in Section 3.5.

A finite exponent bound E allows the synthesis of a contiguous block of primes, but eventually, the bound must be increased (or the partitions must be more exhaustive) to bridge the additive gaps between larger primes.

```
...

fun main() {
    val currentMaxExp = 2
    val maxIterations = 10
    var current = listOf(BigInteger.ONE, BigInteger.valueOf(2))
    var accPrimes = emptyList<BigInteger>()

    for (i in 1..maxIterations) {
        val found = computePrimes(current, currentMaxExp).toList()
        val distinctFound = found.sorted().distinct()

        val currentSet = current.toSet()
        val diff = distinctFound.filter { it !in currentSet }
        val next = if (diff.isEmpty()) current else current + diff.min()

        accPrimes = accPrimes + distinctFound
        current = next
    }
    println("Discovered primes: ${(listOf(BigInteger.valueOf(2)) + accPrimes).
    distinct().sorted()}")
}
```

Listing 2: Modified Main for Constant Exponent Exploration

References

[1] Thomas L. Heath, and Euclid. *The Thirteen Books of Euclid's Elements.*